

17

Lecture 17

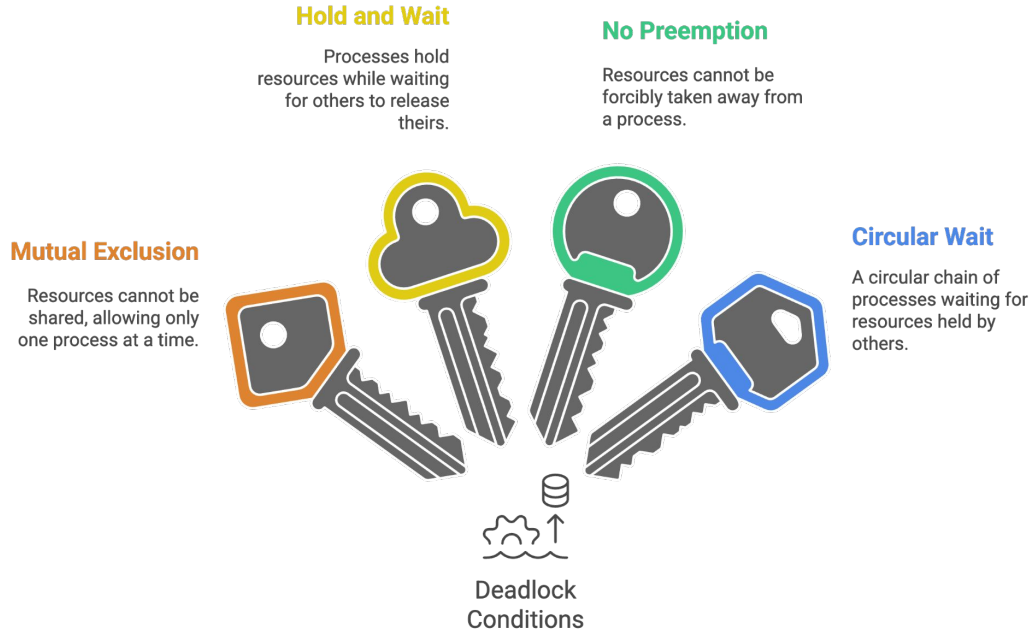
Deadlocks II: Prevention and Avoidance

by - Brijesh Shukla
C07: Operating Systems

The Four Locks of Doom

The Four NECESSARY Conditions

The Discovery: ALL FOUR must exist for deadlock!



Today's Quest: Break at least ONE lock to prevent deadlock forever!

Choose Your Weapon

If you could snap your fingers and eliminate **ONE** of the four deadlock conditions from an operating system, which would you choose?

- A) Mutual Exclusion - Make everything shareable!
- B) Hold and Wait - No holding while waiting!
- C) No Preemption - Take resources by force!
- D) Circular Wait - Impose strict ordering!

- Which seems easiest to break?
- Which would cause least problems?
- Which is your OS superpower choice?



TODAY'S JOURNEY

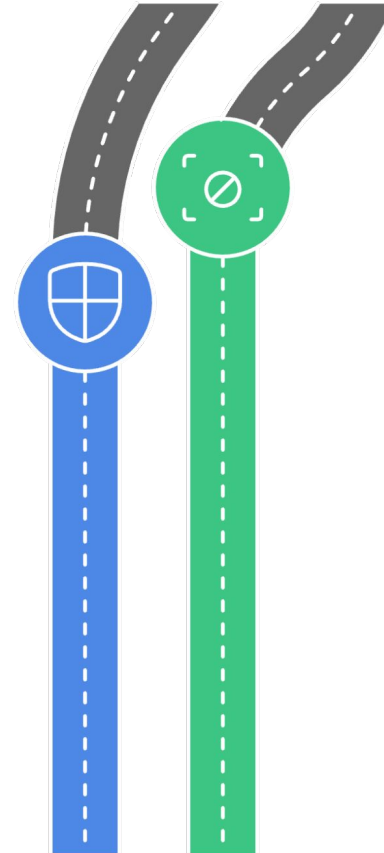
Two Approaches to Defeating Deadlock

Prevention

- Conservative approach
- Break one of the four conditions
- Simple but restrictive
- "Build walls so deadlock can't happen"

Avoidance

- Dynamic approach
- Allow all conditions to exist
- Make smart decisions on-the-fly
- "Navigate carefully to avoid deadlock"



PREVENTION STRATEGY 1

Strategy: Make resources shareable when possible

Examples That Work:

- ✓ Read-only files - Multiple processes read simultaneously
- ✓ macOS Example: Multiple apps reading system config files

The Problem - Not Always Possible:

- ✗ Printers (can't print two jobs simultaneously)
- ✗ Disk blocks being written (data corruption!)
- ✗ Critical sections with shared variables
- ✗ Mutexes (defeats their purpose!)

Reality Check: Some resources are INHERENTLY non-shareable

Verdict: Can't rely on this alone!



PREVENTION STRATEGY 2

Two Approaches:

Approach A: Request All at Once

- Process must request ALL resources before starting
- Get everything or get nothing
- Example: Database transaction locks tables A, B, C all at once

Approach B: Release Before Requesting

- Must release all held resources before requesting new ones
- Start fresh with each request

Advantages:

- ✓ Simple to implement
- ✓ Guarantees no hold-and-wait

Disadvantages:

- ✗ Low resource utilization (hoarding!)
- ✗ Starvation risk (big requests wait forever)
- ✗ Impractical - processes don't know all needs in advance!



Hold and Wait Reality Check

Why might a text editor have difficulty using the "request all resources at once" strategy?

- A) Text editors are too simple
- B) It doesn't know in advance which files the user will open
- C) Text editors don't use resources
- D) The strategy only works for database systems

ANSWER: B) It doesn't know in advance which files the user will open

The text editor can't predict user behavior - which files you'll open, when you'll open them, or how many you'll need. This unpredictability makes "request all at once" impractical for interactive applications.



PREVENTION STRATEGY 3

Strategy: Allow OS to forcibly take resources from processes

Implementation Options:

Option A: If P1 needs resource held by waiting P2, preempt P2

Option B: If P1 can't get resource, preempt ALL its resources

When This Works:

- ✓ CPU time - Just context switch!
- ✓ Memory pages - Swap to disk
- ✓ macOS Virtual Memory - Preempts memory by swapping

When This FAILS:

- ✗ Printers - Can't "undo" half-printed page!
- ✗ Mutexes - State can't be saved/restored
- ✗ Database locks - Transaction consistency breaks
- ✗ File writes - Data corruption risk

The Challenge: Resource state must be savable and restorable



PREVENTION STRATEGY 4

Strategy: Impose total ordering on all resources

The Rule:

- Number all resources: $R_1, R_2, R_3, \dots, R_n$
- Processes **MUST** request in increasing order
- If you hold R_i , you can only request R_j where $j > i$



Why It Works - Proof:

For a cycle to exist: $P_1 \leftarrow R_1 \leftarrow P_2 \leftarrow R_2 \leftarrow P_3 \leftarrow R_3 \leftarrow P_1$

But if P_1 holds R_1 and requests R_3 : R_3 must have number $> R_1$
And if P_3 holds R_3 and requests R_1 : R_1 must have number $> R_3$

CONTRADICTION! Both can't be true!

Therefore: No cycle possible!

Advantages:

- ✓ Simple and provably works
- ✓ Actually used in real systems!

Disadvantages:

- ✗ Must define global ordering (complex in large systems)
- ✗ Forces awkward programming (may need higher-numbered resource first)

PREVENTION TRADE-OFFS SUMMARY

Strategy Comparison

Strategy	Make Shareable	Request All Once	Allow Preemption	Resource Ordering
Breaks	Mutual Exc.	Hold & Wait	No Preemption	Circular Wait
Main Cost	Not always possible	Low utilization, starvation	Limited applicability	Programming burden

Key Insight: Prevention is CONSERVATIVE and PESSIMISTIC

- Assumes deadlock could happen
- Restricts system even when deadlock wouldn't occur
- Trades performance for guaranteed safety
- Often leads to resource underutilization

The Question: Is there a SMARTER way?

Next: Enter Deadlock Avoidance! 

WHEN PREVENTION MAKES SENSE

When to Use Prevention:

✓ Safety-Critical Systems

- Medical devices (pacemakers, surgery robots)
- Aerospace systems
- Nuclear power control
- Cannot afford ANY deadlock risk

✓ Simple Systems

- Few resource types
- Well-understood resource patterns
- Static behavior

✓ Expensive Recovery

- Where detecting/recovering is too costly
- Real-time requirements
- No rollback capability

Example: Kernel locks often use resource ordering

→ Simple, fast, provably safe!



THE PHILOSOPHY SHIFT – AVOIDANCE

Prevention vs Avoidance:

Prevention:

- "Never let dangerous situations be possible"
- Break the fundamental conditions
- Rigid, restrictive
- Like banning all cars from an intersection

Avoidance:

- "Allow danger, but navigate smartly"
- All four conditions CAN exist
- Make intelligent allocation decisions
- Like having smart traffic lights!

The Key Insight:

Just because deadlock COULD happen doesn't mean it WILL!

Avoidance examines each request and asks:

"If I grant this, could deadlock occur later?"

If YES: Make process wait (even if resources available!)

If NO: Grant it immediately



SAFE VS UNSAFE STATES

Key Definitions:

SAFE STATE:

- There exists at least ONE sequence where all processes can finish
- Even if they all request maximum needs RIGHT NOW
- Deadlock is IMPOSSIBLE

UNSAFE STATE:

- No guaranteed safe sequence exists
- Deadlock is POSSIBLE (but not certain!)
- Risky territory

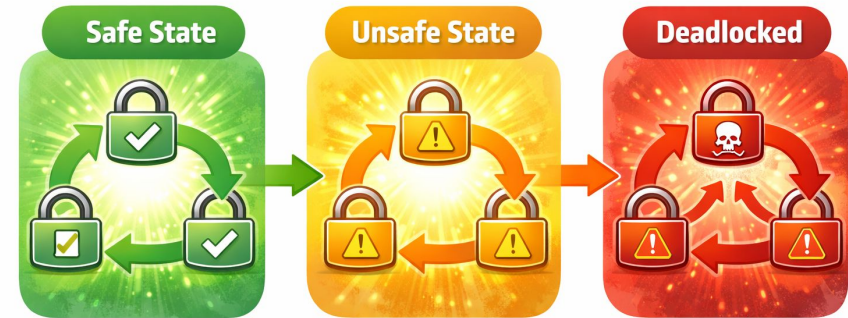
DEADLOCKED STATE:

- Deadlock has already occurred
- Special case of unsafe state

The Relationships:

$[All\ States] \supset [Unsafe\ States] \supset [Deadlocked\ States]$

$Safe \cap Unsafe = \emptyset$ (mutually exclusive)



Avoidance Goal: NEVER enter unsafe states!

Is This Safe?

Scenario: System has 12 tape drives

Three processes with maximum needs:

- P0: max 10 drives (currently has 5)
- P1: max 4 drives (currently has 2)
- P2: max 9 drives (currently has 2)

Current state:

- Allocated: $5 + 2 + 2 = 9$ drives
- Available: $12 - 9 = 3$ drives

Question: Is this SAFE?

Let's analyze...

Analysis:

Available = 3 drives

Step 1: Can P1 finish?

- P1 needs: $4 - 2 = 2$ more drives
- We have: 3 drives available
- YES! ✓ P1 can finish

Step 2: After P1 finishes (releases 2)

- Available = $3 + 2 = 5$ drives

Step 3: Can P0 finish?

- P0 needs: $10 - 5 = 5$ more drives
- We have: 5 drives available
- YES! ✓ P0 can finish

Step 4: After P0 finishes (releases 5)

- Available = $5 + 5 = 10$ drives

Step 5: Can P2 finish?

- P2 needs: $9 - 2 = 7$ more drives
- We have: 10 drives available
- YES! ✓ P2 can finish

Safe Sequence: <P1, P0, P2>

Verdict: SAFE STATE! ✓

Testing Safety Understanding

Same scenario, but we grant P2 one more drive:

- Available: 3 $\rightarrow$ 2 drives
- P2 now has: 2 $\rightarrow$ 3 drives

Is the system still **SAFE**?

- A) Yes, same safe sequence works
- B) No, system is now unsafe
- C) System is deadlocked
- D) Cannot determine without more information

ANSWER: Let's check together:

- P1 needs 2 more, we have 2 $\rightarrow$ P1 finishes, available = 4
 - P0 needs 5 more, we have 4 $\rightarrow$ STUCK! ✗
 - P2 needs 6 more, we have 4 $\rightarrow$ STUCK! ✗
- Result: **UNSAFE!** No safe sequence exists.

INTRODUCING BANKER'S ALGORITHM

Bank Analogy:

- Bank has limited cash (resources)
- Customers have credit limits (max needs)
- Customers take out loans (allocations)
- Banker must ensure: Can always satisfy remaining customers even if all demand at once!

Banker's Algorithm:

- The definitive deadlock avoidance algorithm
- Works with multiple resource types
- Makes smart lending (allocation) decisions
- Ensures system never enters unsafe state

The Promise:

If we follow **Banker's Algorithm**, deadlock is **IMPOSSIBLE!**

Requirements:

1. Each process declares maximum resource needs
2. Processes request incrementally (not all at once)
3. System checks safety before every grant



The Banker's Ledger

For n processes, m resource types:

Available[m]:

- Vector showing currently available resources
- Example: Available = [3, 2, 1] means 3 of R1, 2 of R2, 1 of R3

Max[n][m]:

- Matrix of maximum resource needs
- $\text{Max}[n][m]$ = max of resource type m that process n may need
- Declared in advance!

Allocation[n][m]:

- Matrix of current allocations
- $\text{Allocation}[n][m]$ = resources of type m currently held by process n

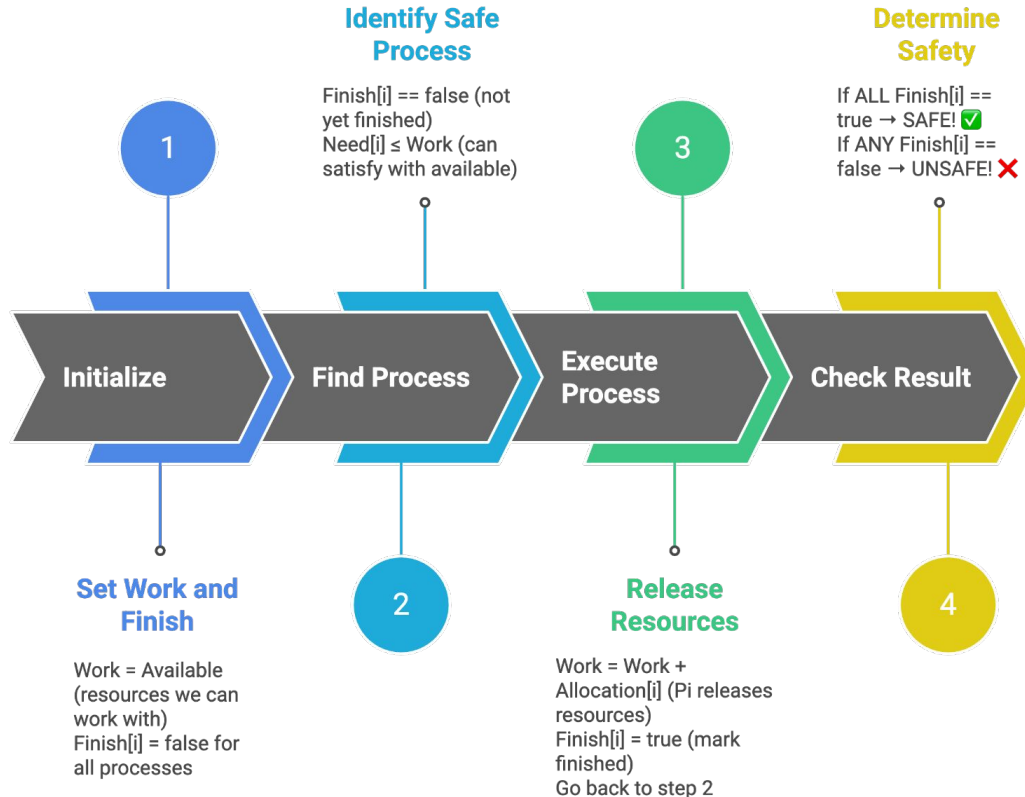
Need[n][m]:

- Matrix of remaining needs
- $\text{Need}[n][m] = \text{Max}[n][m] - \text{Allocation}[n][m]$
- How much MORE each process might need

Key Relationship: Need = Max - Allocation

Can Everyone Be Satisfied?

Safety Algorithm Steps for Deadlock Prevention



RESOURCE REQUEST ALGORITHM

When Process P_i Requests Resources:

$\text{Request}[m]$ = resources being requested

Step 1: Validate Request

- Check: $\text{Request} \leq \text{Need}$
- If not: ERROR (asking more than declared max!)

Step 2: Check Availability

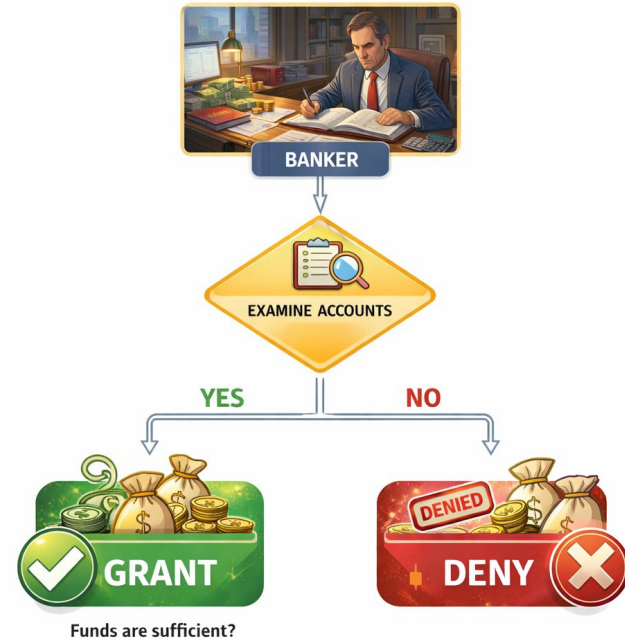
- Check: $\text{Request} \leq \text{Available}$
- If not: P_i must WAIT (resources not available)

Step 3: Pretend to Grant (Tentative Allocation)

- $\text{Available} = \text{Available} - \text{Request}$
- $\text{Allocation}[i] = \text{Allocation}[i] + \text{Request}$
- $\text{Need}[i] = \text{Need}[i] - \text{Request}$

Step 4: Run Safety Algorithm

- If resulting state is SAFE:
 - Grant request permanently ✓
- If resulting state is UNSAFE:
 - Deny request, restore previous state ✗
 - Make P_i wait (even though resources available!)



BANKER'S ALGORITHM SIMULATION

Scenario: 3 processes (P0, P1, P2),
3 resource types (A, B, C)

Given: Available = [3, 3, 2]

Process	Allocation (A B C)	Max (A B C)	Need (A B C)
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0

Tasks:

1. Calculate the Need matrix (Need = Max - Allocation)
2. Determine if current state is SAFE
3. If safe, find a safe sequence
4. If unsafe, explain why

No safe sequence exists!

Step 1: Calculate Need Matrix

Need = Max - Allocation

Now let's check if this state is safe...

Initial: Work = [3, 3, 2], all Finish = false

Round 1: Check each process

- P0: Need [7,4,3] ≤ Work [3,3,2]? NO (7>3) ❌
- P1: Need [1,2,2] ≤ Work [3,3,2]? YES! ✓
→ Finish P1: Work = [3,3,2] + [2,0,0] = [5,3,2]
→ Finish[1] = true

Round 2: Check remaining

- P0: Need [7,4,3] ≤ Work [5,3,2]? NO (7>5) ❌
- P2: Need [6,0,0] ≤ Work [5,3,2]? NO (6>5) ❌

STUCK! Cannot finish P0 or P2!

Result: Finish[0] = false, Finish[2] = false

Verdict: UNSAFE STATE! ⚠️

Banker's Understanding Check

What if Available was [3, 3, 3] instead of [3, 3, 2]?

Would the state be **SAFE**?

- A) Yes, safe sequence exists
- B) No, still unsafe
- C) System is deadlocked
- D) Need more information

ANSWER: B) No, still unsafe

Trace: P1 finishes (Work = [5,3,3]), then P0 needs [7,4,3], we have [5,3,3] - still NO.
But P2 needs [6,0,0] ≤ [5,3,3]? NO (6>5). Still stuck! Actually still UNSAFE.

Detection Frequency Check

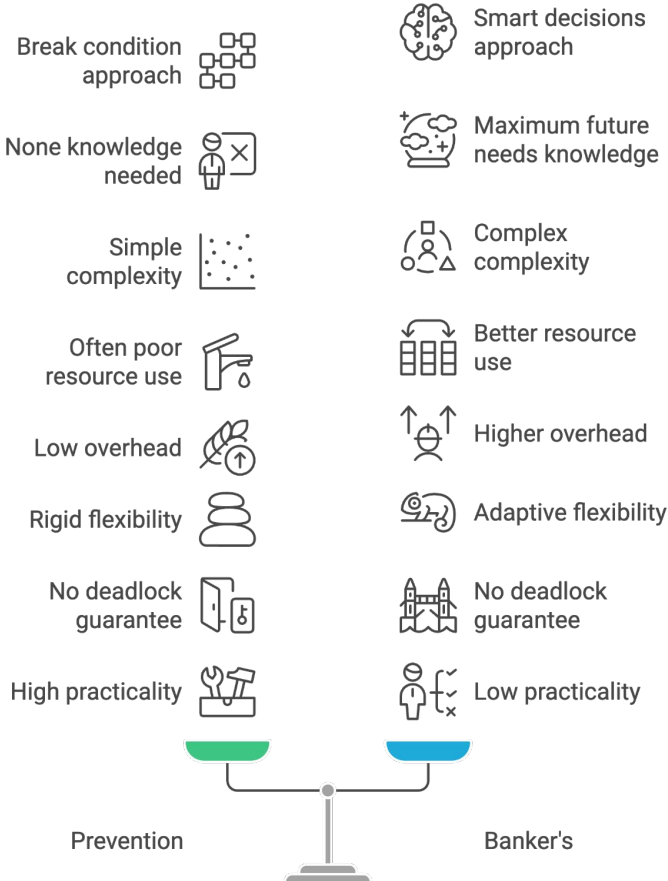
In Banker's Algorithm, the Need matrix represents:

- A) Total resources each process will ever use
- B) Resources currently allocated to each process
- C) Remaining resources each process may still request
- D) Resources available in the system

ANSWER: C) Remaining resources each process may still request

Need = Max - Allocation, showing how much MORE each process might need to complete. It's not the max (that's Max matrix), not current (that's Allocation), and not available (that's Available vector).

BANKER'S VS PREVENTION



Real-World Usage:

- Prevention: Widely used (resource ordering, especially)
- Banker's: Rarely used in practice!

Why Banker's Isn't Used:

- ✗ Processes don't know max needs in advance
- ✗ Number of processes varies dynamically
- ✗ Overhead of safety checks
- ✗ Resources not fixed (devices plug/unplug)

WHY BANKER'S FAILS IN PRACTICE

Real-world scenarios that break Banker's assumptions

Assumption 1: Processes declare max needs in advance

Assumption 2: Fixed number of processes

Assumption 3: Fixed set of resource types

Assumption 4: Processes eventually release resources

Conclusion: Elegant theory, impractical reality

SAFE STATE CHALLENGE

Given state with 3 resource types (A, B, C): Solution:

Available = [2, 1, 0]

Resource	Allocation (A B C)	Max (A B C)	Need (A B C)
P0	0 0 1	2 2 2	2 2 1
P1	1 0 0	3 1 1	2 1 1
P2	1 1 0	2 1 1	1 0 1

Tasks:

1. Verify the Need matrix is correct
2. Is this state SAFE?
3. If safe, provide a safe sequence
4. If unsafe, explain why

1. **Need matrix verification:** ✓ Correct (Max - Allocation)

2. Safety Check:

Work = [2, 1, 0], all Finish = false

Round 1:

- P0: Need [2,2,1] ≤ [2,1,0]? NO (2>1 for B) ✗
- P1: Need [2,1,1] ≤ [2,1,0]? NO (1>0 for C) ✗
- P2: Need [1,0,1] ≤ [2,1,0]? NO (1>0 for C) ✗

STUCK! Cannot finish anyone!

3. Verdict: UNSAFE! ⚠

4. **Why?** Not enough of resource C (0 available) to satisfy any process's remaining needs. All need at least 1 of C.

This state could lead to deadlock!

Key Takeaways

What We've Learned:

1. Prevention: Break **ONE** of Four Conditions

- Mutual Exclusion - Rarely feasible
- Hold and Wait - Impractical for dynamic systems
- No Preemption - Works for some resources
- Circular Wait - Resource ordering WORKS! ✓

2. Resource Ordering is the Winner

- Simple, proven, actually used
- Mathematical guarantee
- Used in real kernels

3. Avoidance: The Smart Approach

- Safe vs Unsafe states
- Banker's Algorithm guarantees safety
- But requires knowing the future...

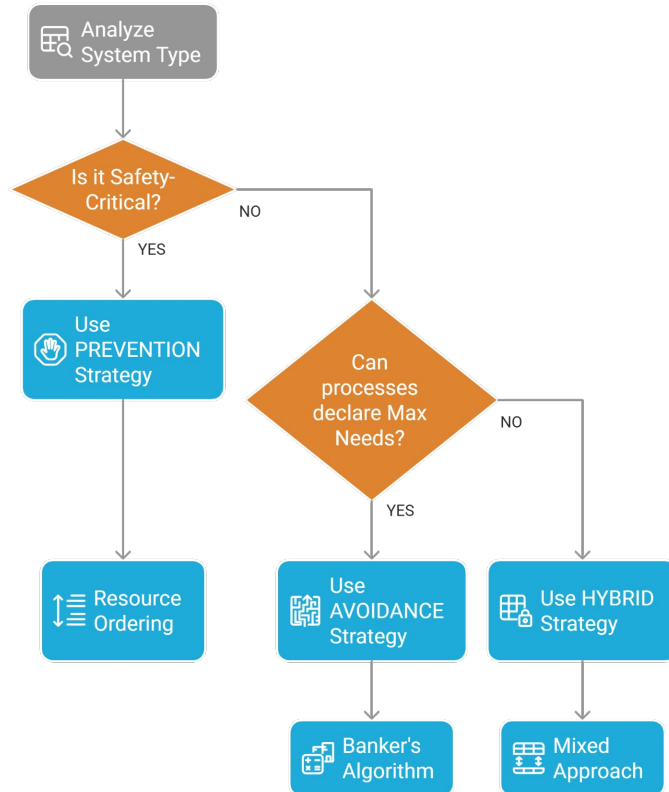
4. Banker's is Beautiful but Impractical

- Theoretical importance
- Rarely used in real systems
- Can't predict future needs



Practical Guidelines

Deadlock Strategy Decision Framework



Modern Best Practices:

1. Resource ordering for kernel locks
2. Lock-free data structures when possible
3. Design patterns that avoid hold-and-wait (GCD!)
4. Timeouts and watchdogs
5. User-initiated recovery (force quit)

Final Understanding Check

What key information must be known in advance for deadlock avoidance to work?

- A) Current CPU utilization
- B) Maximum future resource needs of each process
- C) Priority levels of all processes
- D) Execution time of each process

ANSWER: B) Maximum future resource needs of each process

Avoidance (Banker's Algorithm) requires the Max matrix - each process must declare its maximum resource needs before execution. This lets the algorithm predict worst-case scenarios and maintain safe states.

Thank You!

Thank you for your attention. Feel free to ask questions or revisit the topics covered.